

Learning on Graphs

Homework 2: Spatio-Temporal Forecasting on Graphs

General instructions

This homework studies forecasting of graph signals that evolve over time. You will work with a controlled temporal process on a fixed graph, implement several predictors, and compare temporal-only, graph-only, and spatio-temporal graph neural models.

- Submission is individual.
- Use the provided notebook `hw2_skeleton.ipynb` as the starting point.
- Use Python, NumPy, NetworkX, PyTorch, and Matplotlib. PyTorch Geometric is not required.
- Submit one PDF report and your completed notebook.
- Fix all random seeds and include clear run instructions of your code.

Deliverables

Submit a single zip file containing:

1. `report.pdf`
2. completed `hw2_skeleton.ipynb`
3. `requirements.txt` or short environment notes
4. all figures used in the report, if they are not already embedded in the notebook

Your notebook must run from a clean restart with the default settings.

Point allocation

Component	Points
Part A: Temporal graph dataset and forecasting windows	20
Part B: Forecasting baselines	25
Part C: GCN and spatio-temporal GNN models	35
Part D: Ablations and final comparison	20
Total	100

Dataset

The notebook generates a graph $G = (V, E)$ with approximately $n = 60$ nodes. At time t , the graph signal is

$$x_t \in \mathbb{R}^n.$$

The default temporal process is

$$x_{t+1} = \alpha \tilde{A} x_t + \beta x_t + \gamma u_t + \varepsilon_t,$$

where $\tilde{A}$ is a normalized adjacency matrix with self-loops, u_t is a time-dependent forcing signal on a small set of source nodes, and ε_t is small noise. The default values are

$$T = 400, \quad L = 5, \quad \alpha = 0.45, \quad \beta = 0.35, \quad \gamma = 0.15.$$

If the largest connected component has slightly fewer than 60 nodes, this is fine. Report the final number of nodes.

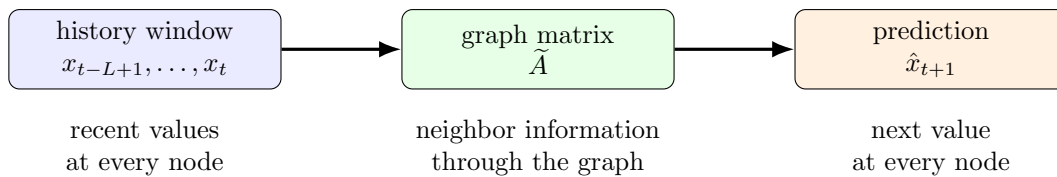


Figure 1: The forecasting problem. Each example contains a short history window and the graph matrix. The target is the next graph signal.

Using the skeleton notebook

The notebook provides the data-generation functions, plotting utilities, chronological window construction interface, training loop, and result-table templates. Complete the cells marked TODO. The model definitions in Parts B and C are intentionally left for you to implement.

Use the default random seed in the notebook unless stated otherwise.

1 Part A: Temporal graph dataset and forecasting windows, 20 points

A1. Generate the temporal dataset, 6 points

Use the notebook to generate:

1. an SBM graph G ,
2. the normalized graph matrix $\tilde{A}$,
3. temporal node signals $x_1, \dots, x_T$ with $T = 400$,
4. a chronological split: 60 percent train, 20 percent validation, 20 percent test.

Report the number of nodes, number of edges, average degree, and the train/validation/test time ranges.

A2. Visualize the data, 8 points

Include the following figures in your report:

1. one visualization of the graph;
2. a line plot of $x_t(i)$ over time for 5 selected nodes;
3. three graph snapshots showing node colors at three different time points.

Use the same layout and color range for all graph snapshots.

A3. Build forecasting windows, 6 points

For a window length L , construct examples

$$H_t = [x_{t-L+1}, x_{t-L+2}, \dots, x_t] \in \mathbb{R}^{n \times L}, \quad y_t = x_{t+1} \in \mathbb{R}^n.$$

Use only windows whose target time belongs to the appropriate split. Report the number of train, validation, and test windows for $L = 5$.

2 Part B: Forecasting baselines, 25 points

Use mean squared error as the main metric:

$$\text{MSE} = \frac{1}{|\mathcal{T}|n} \sum_{t \in \mathcal{T}} \|x_{t+1} - \hat{x}_{t+1}\|_2^2.$$

B1. Persistence baseline, 5 points

Implement

$$\hat{x}_{t+1} = x_t.$$

Report validation and test MSE.

B2. Temporal MLP baseline, 8 points

Implement a node-wise temporal MLP. For each node i , the input is its own last L values:

$$[x_{t-L+1}(i), \dots, x_t(i)] \mapsto \hat{x}_{t+1}(i).$$

Use the same MLP parameters for all nodes. Use $L = 5$, hidden dimension 32, ReLU, Adam, and at most 100 epochs. Report validation and test MSE.

B3. Linear diffusion baseline, 8 points

Implement the graph-aware linear predictor

$$\hat{x}_{t+1} = ax_t + b\tilde{A}x_t + c\mathbf{1}.$$

Fit a, b, c by least squares on the training windows. Report the learned coefficients and validation and test MSE.

B4. Baseline table, 4 points

Include the following table in your report:

Model	Trainable?	Validation MSE	Test MSE
Persistence	no		
Temporal MLP	yes		
Linear diffusion	yes		

3 Part C: GCN and spatio-temporal GNN models, 35 points

This part compares two graph neural predictors. The first model is a GCN forecaster that uses only the current signal x_t . The second model is a spatio-temporal GNN that first extracts temporal features from the history window and then mixes information over the graph.

C1. Implement a dense GCN layer, 7 points

Implement a dense GCN layer without PyTorch Geometric:

$$\text{GCNLayer}(H) = \tilde{A}HW + b,$$

where $H \in \mathbb{R}^{n \times d}$ and $W \in \mathbb{R}^{d \times h}$. Your implementation should support a batch of windows, so the input shape is `batch x nodes x features`.

C2. GCN forecaster, 8 points

Implement a two-layer GCN that receives only the current signal x_t :

$$x_t \in \mathbb{R}^{n \times 1} \rightarrow \text{GCNLayer} \rightarrow \text{ReLU} \rightarrow \text{GCNLayer} \rightarrow \hat{x}_{t+1} \in \mathbb{R}^n.$$

This model uses graph structure but only one time step. Train it with hidden dimension 32, Adam, MSE loss, and at most 100 epochs. Report validation and test MSE.

C3. Spatio-temporal GNN, 12 points

Implement the following model. Given a history window

$$H_t = [x_{t-L+1}, \dots, x_t] \in \mathbb{R}^{n \times L},$$

first apply a temporal convolution independently at every node:

$$U_t(i, :) = \text{TemporalConv}_\theta(H_t(i, :)) \in \mathbb{R}^h.$$

Then apply graph mixing:

$$Z_t = \text{ReLU}(\tilde{A}U_tW_1 + b_1), \quad \hat{x}_{t+1} = \tilde{A}Z_tw_2 + b_2.$$

Use a one-dimensional convolution over the length- L history with kernel size 3, padding 1, hidden dimension 32, and mean pooling over the temporal dimension after the convolution. The same temporal convolution is shared by all nodes.

C4. Train and compare the GCN and spatio-temporal GNN, 8 points

For $L = 5$, train both graph neural models using the same train/validation/test split. Include:

1. number of trainable parameters for each model;
2. training and validation loss curves;
3. validation and test MSE.

4 Part D: Ablations and final comparison, 20 points**D1. History-length ablation, 7 points**

Train the spatio-temporal GNN with

$$L \in \{1, 3, 5, 10\}.$$

Report validation and test MSE for each value. Use the same hidden dimension and epoch budget for all runs.

D2. Graph ablation, 6 points

Repeat the $L = 5$ spatio-temporal GNN experiment after replacing $\tilde{A}$ with the identity matrix I . This removes graph mixing while keeping the same temporal part of the model.

Include this table:

Model	Window L	Graph matrix	Test MSE
GCN forecaster	1	$\tilde{A}$	
Spatio-temporal GNN	1	$\tilde{A}$	
Spatio-temporal GNN	3	$\tilde{A}$	
Spatio-temporal GNN	5	$\tilde{A}$	
Spatio-temporal GNN	10	$\tilde{A}$	
Spatio-temporal GNN	5	I	

D3. Final comparison and conclusions, 7 points

Create one compact table comparing the main models:

Model	Validation MSE	Test MSE
Persistence		
Temporal MLP, $L = 5$		
Linear diffusion		
GCN forecaster		
Spatio-temporal GNN, best L		
Spatio-temporal GNN, $L = 5$, no graph		

Then answer the following in short paragraphs, using your measured numbers:

1. Which model performs best on the test set?

2. Did temporal history help beyond using only x_t ?
3. Did graph structure help beyond node-wise temporal modeling?

Each answer should be 3 to 5 sentences.

Report checklist

Your PDF report should include:

1. graph statistics and split ranges;
2. graph visualization, node time-series plot, and graph-signal snapshots;
3. baseline table;
4. GCN and spatio-temporal GNN training curves;
5. GCN versus spatio-temporal GNN comparison;
6. history-length ablation table;
7. graph ablation table;
8. final comparison table;
9. short result-based conclusions.